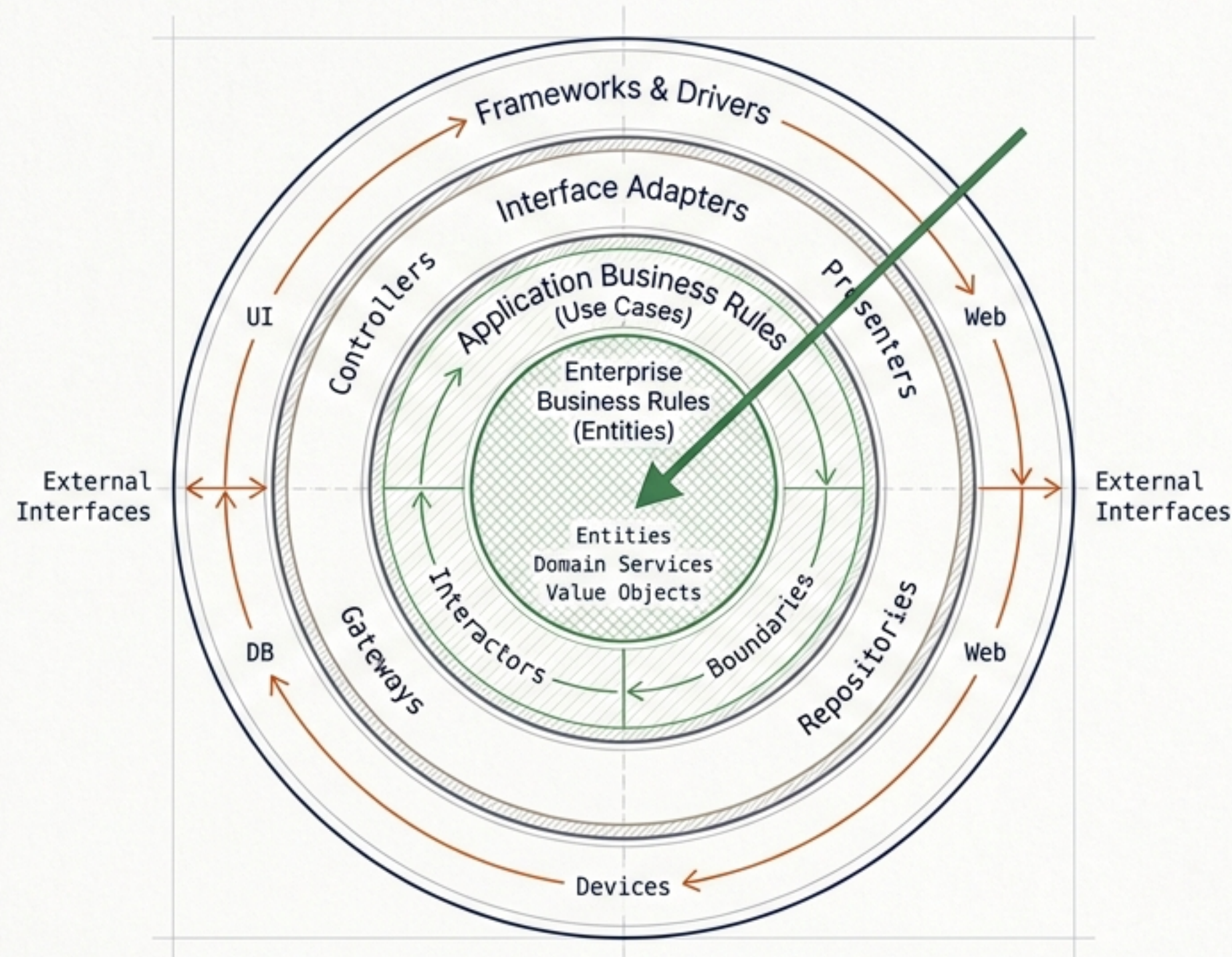




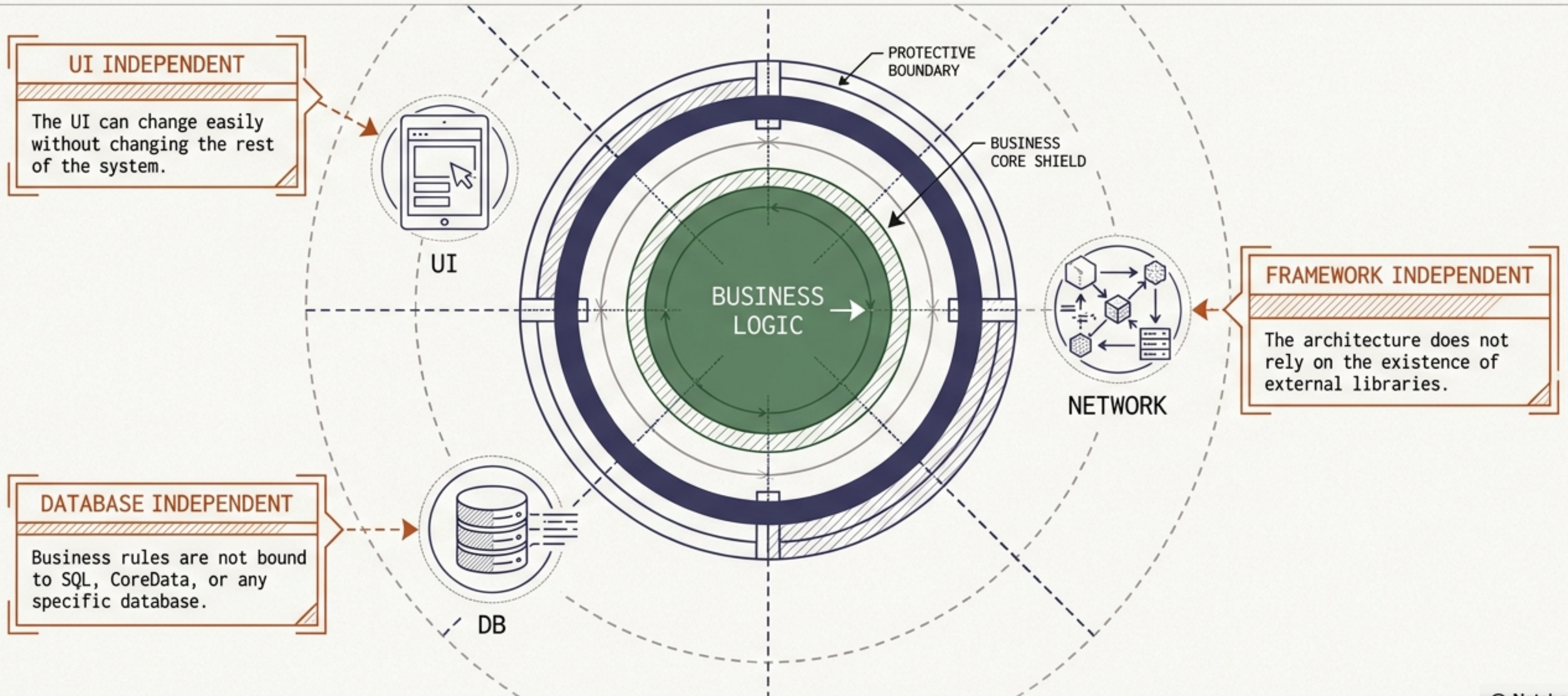
The Clean Architecture Blueprint

Decoupling frameworks, isolating business logic, and engineering software for scale.

A visual deconstruction of the design philosophy popularized by Robert C. Martin

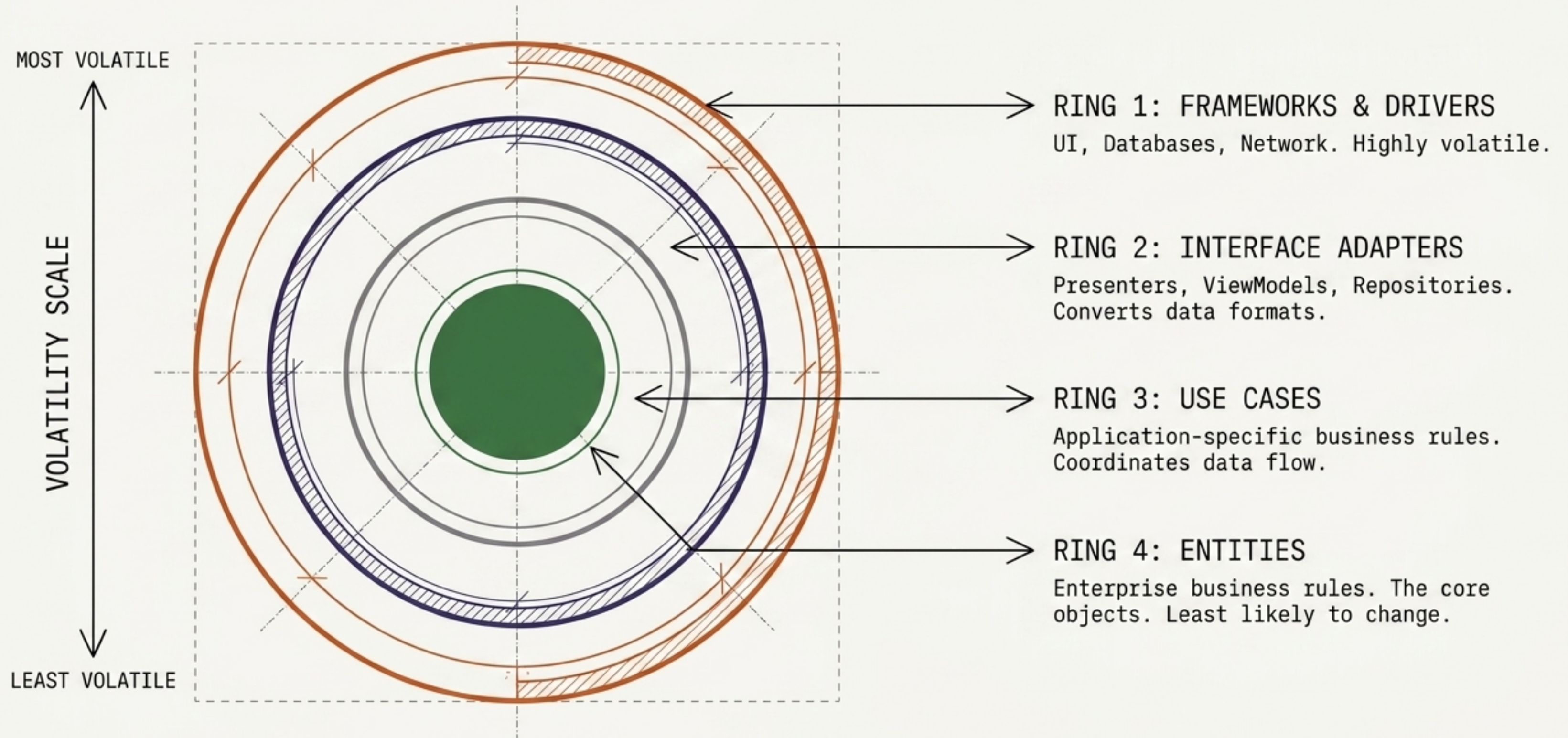
TOTAL ISOLATION OF THE BUSINESS CORE

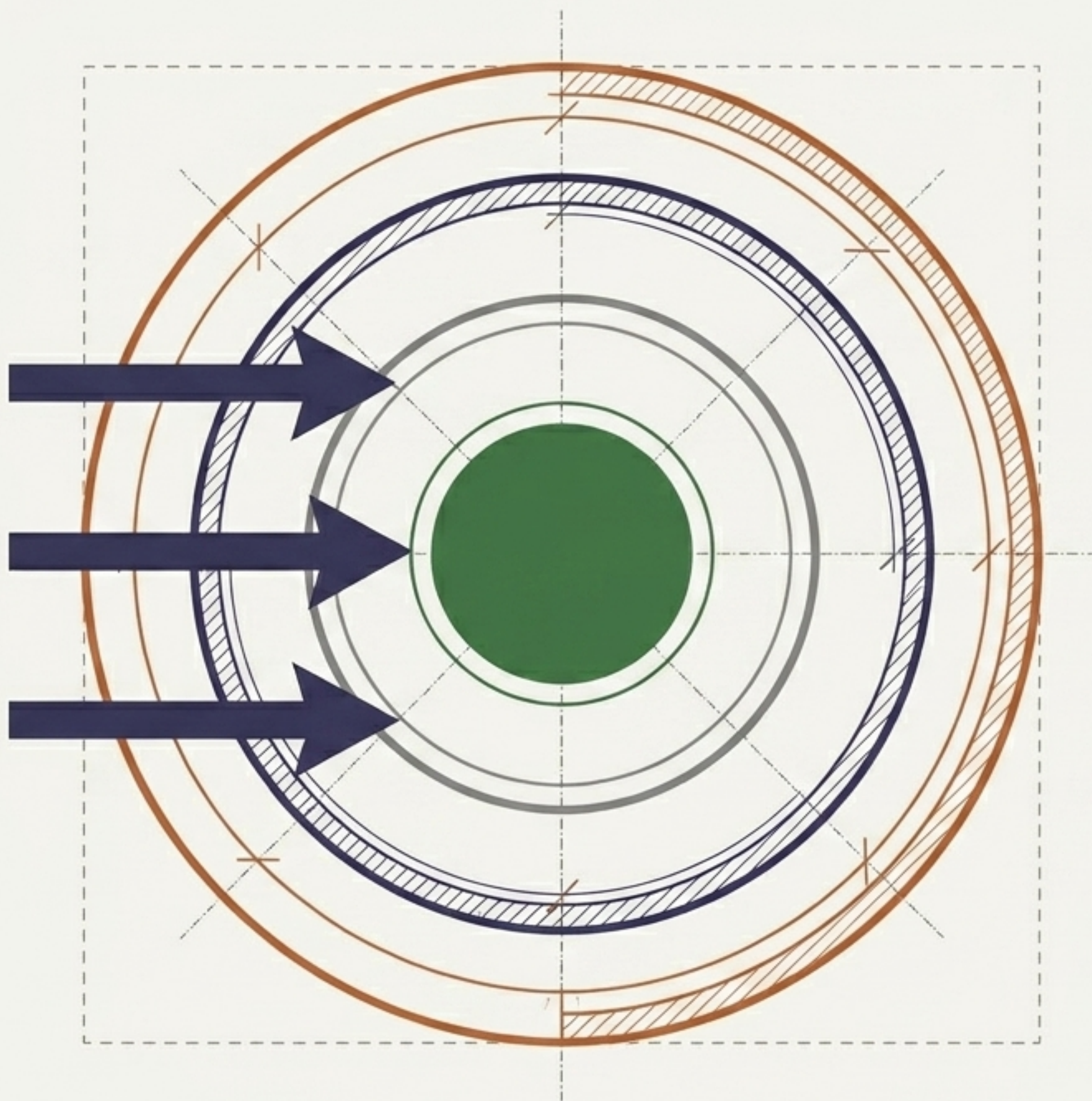
The primary mandate of Clean Architecture is the separation of concerns. It isolates the fundamental business rules of an application from the mechanisms of delivery and storage.



THE LAYERS OF ABSTRACTION

These apprenaitary innction diam slires that had syeed an architecture, alter glowing nodes, and dark mode, alter. footer elements.





Dependencies must point inward

Source code dependencies can only point inward. Nothing in an inner circle can know anything at all about something in an outer circle.

Entity

<- knows nothing about
<- Database

Use Case

<- knows nothing about
<- ViewModel



Forbidden: Outward
Dependencies

THE DATA FLOW PARADOX

EXECUTION DATA FLOW

1. VIEW
NOTIFIES
VIEWMODEL

2. VIEWMODEL
EXECUTES
USE CASE

3. USE CASE
ASKS REPOSITORY
FOR DATA

4. REPOSITORY
FETCHES FROM
DATA SOURCE

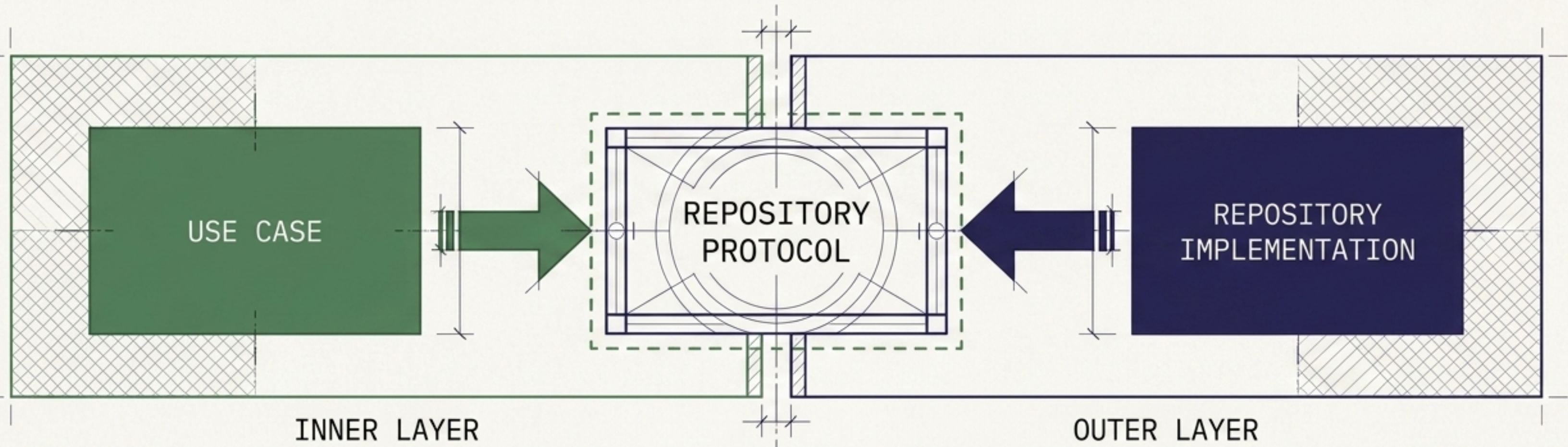
5. DATA
RETURNS AS
ENTITIES OR DTOS

THE DEPENDENCY RULE

PARADOX: How does the Use Case ask the Repository for data without pointing an outward dependency at it?

Dependency Inversion bridges the boundary

- The Use Case calls the Protocol (Interface).
- The Protocol lives in the same inner layer as the Use Case.
- The Repository (outer layer) implements the Protocol.

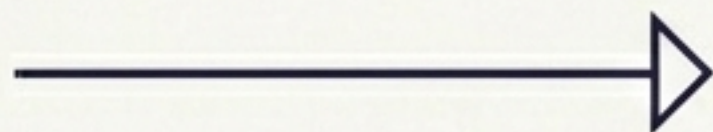


Insight: By using an interface, the flow of control moves outward to fetch data, but the source code dependency points inward toward the interface. The rule remains unbroken.

Production example: A Swift iOS User List



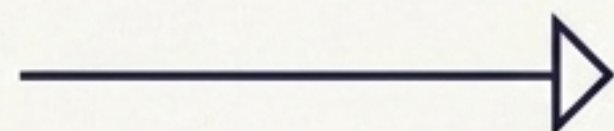
Domain/



Contains zero dependencies on external frameworks or networking libraries. Pure business logic.



Data/



Handles the actual fetching. Knows about APIs and Databases.



Presentation/



Uses the Use Case to get data and formats it for the View.

Mapping the codebase to the blueprint



The architectural ROI matrix

Benefit	The Architectural Enabler		Practical Impact	
Testability	→	Interface Adapters & Dependency Inversion	→	Test Use Cases and ViewModels instantly by mocking the Repository protocols without hitting a real API or DB.
Independence	→	Frameworks Layer Isolation	→	Swap URLSession for Alamofire, or CoreData for Realm, without ever touching the core business logic.
Scalability	→	Strict Boundary Crossing	→	Multiple developers can work simultaneously with minimal merge conflicts.

Synthesis: The full lifecycle of a request

